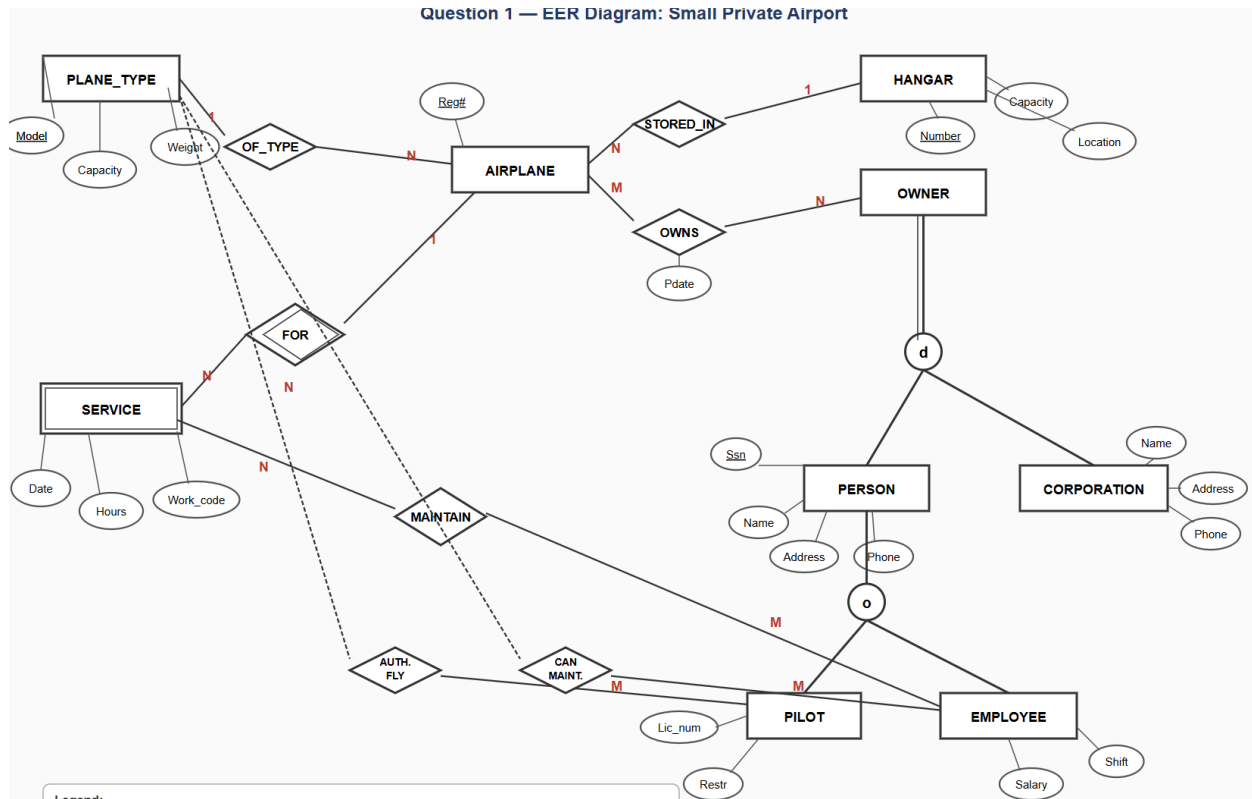


CS 5200 Database Management Systems

Question 1 — EER Diagram: Small Private Airport (10 pts)\



Question 2 — Requirements from Airline ER Diagram (10 pts)

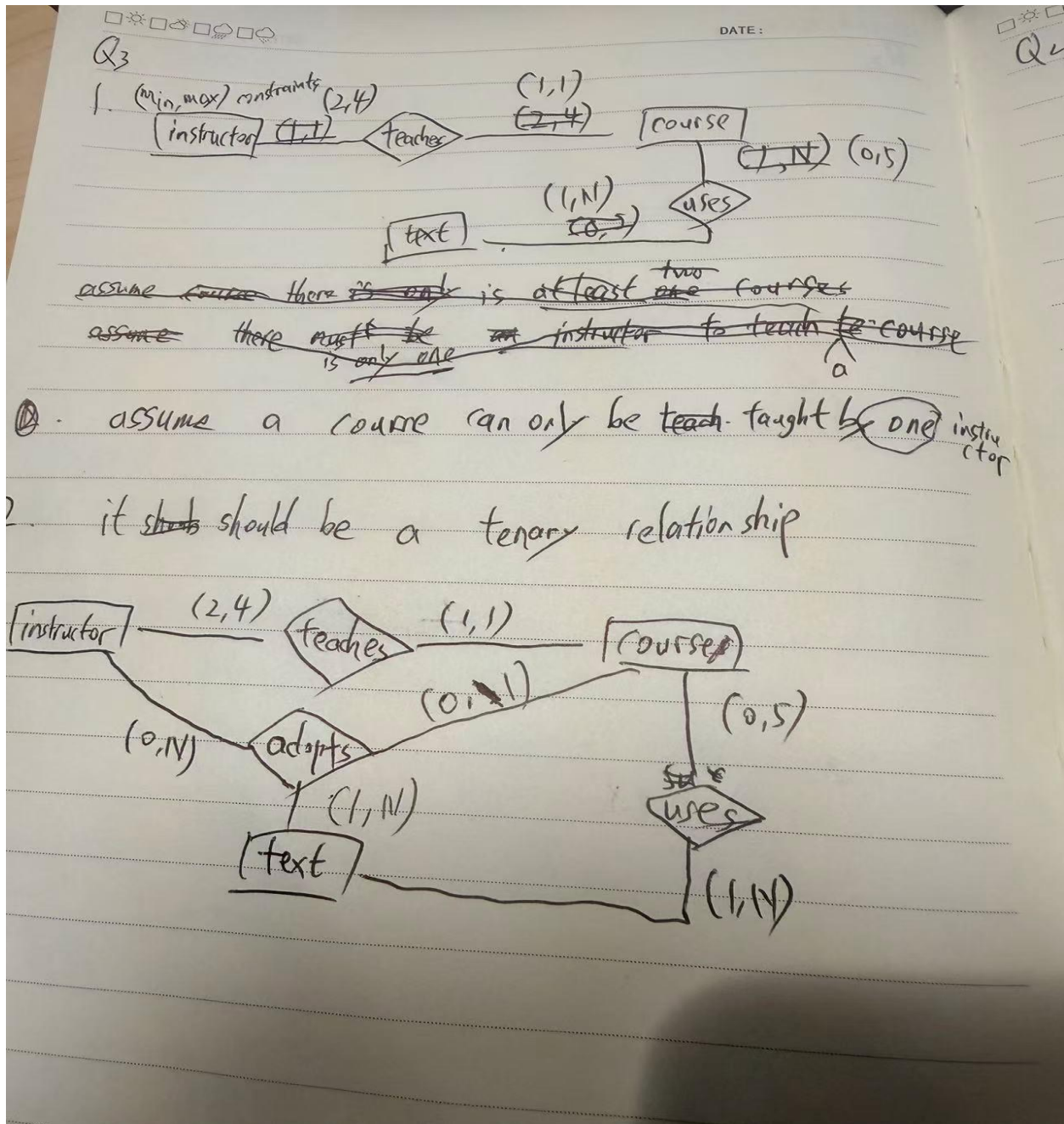
Entities & Attributes

- AIRPORT: Airport_code (PK), City, State, Name
- FLIGHT_LEG: Leg_no (PK), Scheduled_dep_time, Scheduled_arr_time
- FLIGHT: Number (PK), Airline, Weekdays
- AIRPLANE_TYPE: Type_name (PK), Max_seats, Company
- AIRPLANE: Airplane_id (PK), Total_no_of_seats
- LEG_INSTANCE: Date (PK), No_of_avail_seats, Arr_time
- FARE: Code (PK), Amount, Restrictions
- SEAT: Seat_no (PK)
- RESERVATION: Customer_name, Cphone

Requirements

1. Each airport has a unique code, and is located in a city and state.
2. A flight leg has a scheduled departure and arrival time, and connects exactly one departure airport (1:N) to one arrival airport (1:N).
3. An airport can serve as a departure airport for many legs, and as an arrival airport for many legs.
4. A flight consists of one or more legs (LEGS relationship, 1:N).
5. Each flight leg can have many instances — one per date (INSTANCES, 1:N).
6. Each airplane type can be flown from many airports (CAN_LAND, M:N).
7. Each airplane belongs to exactly one airplane type (TYPE, N:1).
8. An airplane is assigned to leg instances (ASSIGNED, N:M — an airplane can cover many instances, an instance uses one airplane).
9. An airplane departs from an airport (DEPARTS, with Dep_time) and arrives at an airport (ARRIVES, with Arr_time).
10. A leg instance can have many fares (FARES, 1:N); each fare belongs to one instance.
11. A leg instance can have many reservations; each reservation is for exactly one instance (1:N).
12. A reservation can include many seats; a seat belongs to exactly one reservation (N:1).

Question 3 — (min,max) Constraints & ADOPTS Relationship (10 pts)



Question 4 — Generalization EER Diagram (10 pts)

Identifying the Generalization

Looking at Figure 3, INSTRUCTOR and STUDENT share common attributes such as Id/Ssn and Name. Applying generalization, we extract a superclass UNIVERSITY_PERSON.

Superclass

- UNIVERSITY_PERSON: Id (PK), Name

Subclasses

- INSTRUCTOR (inherits Id, Name) + specific attributes: Rank, IOffice, IPhone
- STUDENT (inherits Id, Name) + specific attributes: SId, FName, MName, LName, DOB, SName, Addr, Phone, Major

Specialization Constraints

- Completeness: Partial — not every university person must be an instructor or student (e.g., a dean may be classified separately)
- Disjointness: Overlapping (o) — a person could be both an instructor and a student

Question 5 — SQL DDL for LIBRARY Database (10 pts)

Create Tables (in dependency order)

```
-- 1. PUBLISHER (no foreign keys)
CREATE TABLE PUBLISHER (
    Name      VARCHAR(50)  PRIMARY KEY,
    Address   VARCHAR(100),
    Phone     VARCHAR(20)
);

-- 2. LIBRARY_BRANCH (no foreign keys)
CREATE TABLE LIBRARY_BRANCH (
    Branch_id  INT          PRIMARY KEY,
    Branch_name VARCHAR(50)  NOT NULL,
    Address    VARCHAR(100)
);

-- 3. BORROWER (no foreign keys)
CREATE TABLE BORROWER (
    Card_no INT          PRIMARY KEY,
    Name     VARCHAR(50)  NOT NULL,
    Address  VARCHAR(100),
    Phone    VARCHAR(20)
);

-- 4. BOOK (references PUBLISHER)
CREATE TABLE BOOK (
    Book_id      INT          PRIMARY KEY,
    Title         VARCHAR(100) NOT NULL,
    Publisher_name VARCHAR(50),
    FOREIGN KEY (Publisher_name) REFERENCES PUBLISHER(Name)
        ON DELETE SET NULL ON UPDATE CASCADE
);

-- 5. BOOK_AUTHORS (references BOOK)
CREATE TABLE BOOK_AUTHORS (
    Book_id      INT,
    Author_name  VARCHAR(50),
    PRIMARY KEY (Book_id, Author_name),
    FOREIGN KEY (Book_id) REFERENCES BOOK(Book_id)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- 6. BOOK_COPIES (references BOOK, LIBRARY_BRANCH)
CREATE TABLE BOOK_COPIES (
    Book_id      INT NOT NULL,
```

```

Branch_id    INT NOT NULL,
No_of_copies INT DEFAULT 0,
PRIMARY KEY (Book_id, Branch_id),
FOREIGN KEY (Book_id) REFERENCES BOOK(Book_id)
    ON DELETE CASCADE ON UPDATE CASCADE,
FOREIGN KEY (Branch_id) REFERENCES LIBRARY_BRANCH(Branch_id)
    ON DELETE CASCADE ON UPDATE CASCADE
);

-- 7. BOOK_LOANS (references BOOK, LIBRARY_BRANCH, BORROWER)
CREATE TABLE BOOK_LOANS (
    Book_id    INT NOT NULL,
    Branch_id  INT NOT NULL,
    Card_no    INT NOT NULL,
    Date_out   DATE,
    Due_date   DATE,
    PRIMARY KEY (Book_id, Branch_id, Card_no),
    FOREIGN KEY (Book_id) REFERENCES BOOK(Book_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (Branch_id) REFERENCES LIBRARY_BRANCH(Branch_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (Card_no) REFERENCES BORROWER(Card_no)
        ON DELETE CASCADE ON UPDATE CASCADE
);

```

ON DELETE CASCADE → child rows are deleted when the parent row is deleted.
ON DELETE SET NULL → child FK becomes NULL when the parent row is deleted.
Always build parent tables before child tables!

Question 6 — SQL Queries on Student Database (10 pts)

Query 1 — Straight-A Students (grade A in ALL courses)

Logic: find students for whom there does NOT EXIST any grade record that is not 'A'.

```
SELECT S.Name, S.Major
FROM   STUDENT S
WHERE  NOT EXISTS (
    SELECT *
    FROM   GRADE_REPORT GR
    WHERE  GR.Student_number = S.Student_number
          AND GR.Grade <> 'A'
);
```

Key logic: NOT EXISTS + Grade <> 'A'

= There is no course where the grade is not A

= All courses have grade A

Common mistake: WHERE Grade = 'A' only finds students with AT LEAST one A, not ALL A.

Query 2 — Students with No A at All

Logic: find students whose student number does not appear in any 'A' grade record.

```
SELECT S.Name, S.Major
FROM   STUDENT S
WHERE  S.Student_number NOT IN (
    SELECT GR.Student_number
    FROM   GRADE_REPORT GR
    WHERE  GR.Grade = 'A'
);
```

Question 7 — SQL DML on COMPANY Database (20 pts)

1. Insert new employee Robert Scott

```
INSERT INTO EMPLOYEE
    (Fname, Minit, Lname, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)
VALUES
    ('Robert','F','Scott','943775543','1972-06-21',
    '2365 Newcastle Rd, Bellaire, TX','M',58000,'888665555',1);
```

2. Insert new project ProductA

```
INSERT INTO PROJECT (Pname, Pnumber, Plocation, Dnum)
VALUES ('ProductA', 4, 'Bellaire', 2);
```

3. Insert new department Production

```
INSERT INTO DEPARTMENT (Dname, Dnumber, Mgr_ssn, Mgr_start_date)
VALUES ('Production', 4, '943775543', '2007-10-01');
```

Note on insert order: DEPARTMENT must be inserted before EMPLOYEE if Dno is a FK to DEPARTMENT.

Here Dno=1 already exists, so the employee insert is safe.

Always satisfy FK constraints before inserting child rows.

4. Delete WORKS_ON tuples where Essn = '333445555'

```
DELETE FROM WORKS_ON
WHERE Essn = '333445555';
```

5. Update manager of Department 5

```
UPDATE DEPARTMENT
SET     Mgr_ssn          = '123456789',
        Mgr_start_date = '2007-10-01'
WHERE  Dnumber = 5;
```


Question 8 — SQL Queries on Student Database (20 pts)

Query 1 — Senior CS students (Class = 4)

```
SELECT Name
FROM STUDENT
WHERE Major = 'CS'
      AND Class = 4;
```

Query 2 — Sections taught by King with student count

```
SELECT S.Course_number,
       S.Semester,
       S.Year,
       COUNT(GR.Student_number) AS Num_students
FROM SECTION S
LEFT JOIN GRADE_REPORT GR
      ON S.Section_identifier = GR.Section_identifier
WHERE S.Instructor = 'King'
GROUP BY S.Section_identifier,
         S.Course_number,
         S.Semester,
         S.Year;
```

Why LEFT JOIN? — To include sections where no students are enrolled yet (COUNT = 0).
Why GROUP BY multiple columns? — SELECT contains Course_number, Semester, Year;
all non-aggregate columns in SELECT must appear in GROUP BY.

Query 3 — Straight-A students (same logic as Q6 Query 1)

```
SELECT S.Name, S.Major
FROM STUDENT S
WHERE NOT EXISTS (
    SELECT *
    FROM GRADE_REPORT GR
    WHERE GR.Student_number = S.Student_number
          AND GR.Grade <> 'A'
);
```

Query 4 — Insert new student Johnson

```
INSERT INTO STUDENT (Name, Student_number, Class, Major)
VALUES ('Johnson', 25, 1, 'Math');
```

Query 5 — Change Smith's class to 2

```
UPDATE STUDENT
SET   Class = 2
WHERE Name  = 'Smith';
```

Quick Reference Summary

Q#	Points	Topic	Key Concept
Q1	10	EER Diagram (from text)	Weak entity, inheritance, relationship attributes
Q2	10	Requirements (from diagram)	Reverse-engineer ER diagram into text
Q3	10	(min,max) + Ternary	ADOPTS needs 3 entities; (min,max) per entity
Q4	10	Generalization EER	Extract superclass from common attributes
Q5	10	SQL DDL	CREATE TABLE + PRIMARY KEY + FOREIGN KEY + triggers
Q6	10	SQL Queries	NOT EXISTS + $\forall$ 'A' for universal quantification
Q7	20	SQL DML	INSERT / DELETE / UPDATE with WHERE
Q8	20	SQL Comprehensive	JOIN + GROUP BY + COUNT + NOT EXISTS